Republic of Tunisia
Ministry of Higher Education
and Scientific Research

University of Tunis El Manar

Higher Institute of Computer Science

Operating system mini project report

Carried out by
Asma NEJI

Student in
Computer engineering

Specialty: Engineering and Development of Communications Infrastructures and Services

Creating a shell command interpreter

Teacher:

Mrs Yousra Najar

Academic year: 2022 - 2023

Contents

List of Figures

General introduction

This report will be dedicated to the creation of a C program that will simulate the Shell command interpreter. To do this, several functions and standard libraries must be used. In this purpose, I will be using the fork(), open(), exec() and wait() system calls, but without using the system() call.

Thereafter, my report will be divided into two parts: the first part is to explain the entry point of my software and the data structures, as well as to specify the algorithms made to realize my program, and the second part will comprise a guide to installing and using the software.

Chapter 1

Data structures and algorithms

1.1 Introduction

In this part, I will list the functionalities produced by my Shell, as well as talk about the data structures and algorithms used for the realization of this command interpreter.

1.2 Shell Features

When creating my Shell, I used a single C file containing functions and procedures. Similarly, I used arrays and linked lists to structure my code. In my code I tried to produce the most important features that a shell must have. For the creation of my Shell, I tried to make the following functionalities:

1.2.1 Display current directory

When the user opens the command interpreter, the first thing that should be displayed is the current directory. For the realization of this functionality, I use the "getcwd" function. This function is used to read the current directory. Then I used "printf" which displays the current directory to the user.

1.2.2 Bach mode

Since there are two execution modes: interactive mode and batch mode. I fixed both modes to improve my program. Batch mode is for executing commands in a file. First, the file will be open, if the file does not exist, we must return an error message and exit. We must read the file line by line with the fgets function. If we encounter the "#" character, we continue execution, otherwise, we exit the file.

1.2.3 Interactive mode

We associate the "%" character to the prompt variable, which will be used as the command prompt, then we use the printf function to display it to the user. We use the fgets function to read the commands typed by the user and return to the line to read the next command. We use the strlen function to measure the length of the string, if it is less than 0, we display

an error message. We use the strcmp function to find out if the string "quit" has been typed by the user. In this case, we leave Shell.

1.2.4 Simple command

For the execution of simple commands, we must create a process with the fork system call and the child that will be created by duplication must be executed with execvp.

1.2.5 Compound command

We use the strsep command to separate the entries of the command according to the separator "&&", "||" and ";". If there is one of these separators in the line, then we execute each command separately. For this system call, we need two forks, therefore two threads. To execute commands according to "&&" separator, we must execute the second command only if the first command is being executed. To execute commands according to "||" separator, we must execute the second command only if the first command is failed executing. And finally, to execute commands according to ";" we must execute the first command and then execute the second command.

1.2.6 Redirect standard output

Redirect to a pipe

We use the strsep command to separate the entries of the command according to the separator "|". This function is used to redirect the standard output of commands to an unnamed pipe. If there is no such separator, we return 0. If we have found the separator, we must make a call to two forks so there will be two threads to execute the commands.

Redirect to a file

For file redirection, we need the ">" command, we try to read each word of the command and compare the first character of each word to this ">" character. If there is a match, the destination file is then opened. Similarly, there will be a call for a fork, therefore a child. Otherwise, an error message is displayed.

1.2.7 Error message

An error message will be displayed if:

- Batch file does not exist: if the file does not exist, we must return an error and exit as I had described in the "batch mode" section.

- An inadequate number of arguments in the command prompt: For this, we use the strlen function as I described in the "interactive mode" section.

- A command does not exist or cannot be executed: If the process identifier is equal to 0.

1.2.8 History

First, we test if the length of the line is greater than 0, if so, we add the typed command to the history.

1.2.9 Libraries used:

The following libraries were used for calling functions.
#include <sys/wait.h>
waitpid()
#include <unistd.h>
fork() / exec() / pid_t / getcwd() / pipe()
#include <stdlib.h>
free() / exit() / execvp() / EXIT_FAILURE / fclose() / fopen() / close() / dup2() / wait()
#include <stdio.h>
fprintf() / printf() / stderr / fgets()
#include <string.h>
strcmp() / strtok() / strlen() / strsep()
#include<readline/readline.h>
#include<readline/history.h>
readline() / add_history()

1.3 Conclusion

In this part, I described the functionalities used in my program. In the next part, I will be guiding you throw the software installation.

Chapter 2

Software Installation and User Guide

2.1 Introduction

So that other users can benefit from my software, it is essential to describe the steps of its installation and use. In this part, I will guide users into these steps.

2.2 Software installation:

I used makefile to automatically build my shell program. The following code shows a simple makefile to automate the production of the executable, the installation of my software and the removal of the executable.

Makefile:

```
prog: progV2.o
    gcc -o prog progV2.o

progV2.o : progV2.c
    gcc -c -Wall progV2.c

install :
    ./configure
    make
     make install

clean :
    rm -f prog *.o
```

These instructions work as expected on occasion, but sometimes make install gets in the way, because the software expects to be able to write to /usr/local or some other filesystem shared space. The sudo make install command will always cause the procedure to terminate because sudo requires administrator (root) permissions. The solution is to put a –prefix flag in the configure step so that the installation is directed to the directory of your choice, for example:

```
./configure --prefix=/my/project/directory/some-package && make && make install
```

2.3 Using the software:

Once the software is properly installed, user can start trying to type some basic commands to test the Shell.

Firstable, when the user open the software, the first thing will be displayed is the figure below:

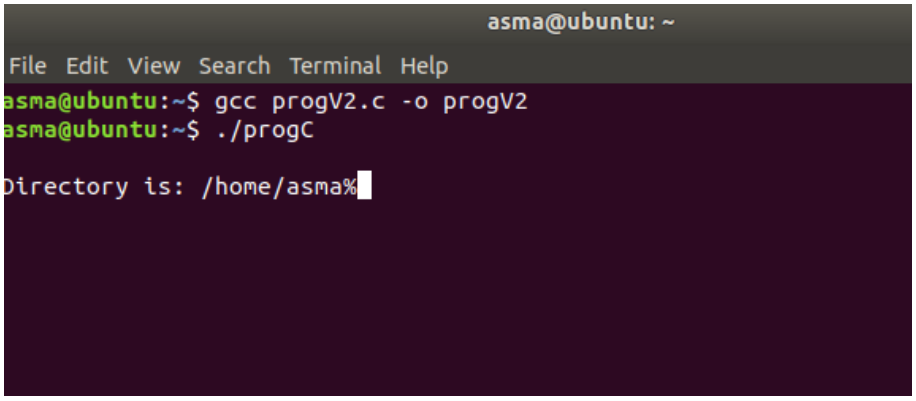

Figure 2.1: User interface

To test my Shell, you can use the basic commands listed below:
- pwd: Writes to standard output the full path name of your current directory.
- cd: Move to a directory: cd rep1.
- ls: List the files and folders of a directory: ls dossier1.
- touch: Create a file: touch file1.
- mkdir: Create a directory: mkdir rep1.
- rm: Delete an empty file: rm file1.
- rm: Delete an empty directory: rm rep1.
- cp: Copies a file file1 into a file file2: cp file1 file2.
- cp: Copies the directory rep1 in a directory rep2: cp -R rep1 rep2.
- mv: Move file file1 to another file file2: mv file1 file2.
- find: Search for files containing a string in a directory: find rep1 -name "training-it".

Commands for displaying the contents of a file
- cat: Displays the contents of a file: cat file1
- more: Displays the contents of a file by a page: more file1
- tail: Displays the last 20 lines of a file: tail -20 file1
- head: Displays the first 20 lines of the file: head -20 file1
- grep: Searches for a character string in a file: grep "training" file1.txt

2.4 Testing the software:

To test all the functionalities provided by the software, you can use these commands below:

Testing single command

ls

Testing and operator

pwd && ls

Testing or operator

pwd || ls

Testing newline separator

pwd ; ls

Testing pipe

ls /home/ | wc -b

Testing redirect operator

ls > file.txt

Testing history

history

These are some examples of commands a have used to test my shell:

```
Directory is: /home/asma
% ls
a.out    docu    Documents  examples.desktop  Music     prog1    progC
Desktop  docu.c  Downloads  long.c            Pictures  prog1.c  progC.c

Directory is: /home/asma
% pwd
/home/asma

Directory is: /home/asma
% pwd || ls
/home/asma
```

Figure 2.2: First test

```
Directory is: /home/asma
% ls > test
ls: cannot access '>': No such file or directory
test:

Directory is: /home/asma
% ls > file.txt
ls: cannot access '>': No such file or directory
ls: cannot access 'file.txt': No such file or directory
```

Figure 2.3: Second test

2.5 Conclusion

This part was a brief introduction of how to install the software with makefile and use it.

General conclusion

All along this report, I tried to explain the important parts for the use of the software and its functionalities. This had required me to call several C functions, create functions and procedures, and then reintroduce them into the int main() function. After all, I have learned a little bit about how to create a mini Linux Shell.It would take a lot of work to make a professional shell.